

Beam avanzado: ventanas, estado y efectos externos

Streaming de datos y sus aplicaciones

Maestría en Inteligencia Artificial

Facultad Politécnica - Universidad Nacional de Asunción

Rodrigo Parra, M.Sc.

Clase 6

Contents

1. Del grafo al comportamiento temporal

- La clase 5 construyó el grafo; hoy controlamos su memoria
- Hoy procesaremos pagos que llegan tarde o duplicados
- Tres fallos cambian un total aparentemente simple
- Resultados de aprendizaje
- El diseño responde cuatro preguntas antes del código

2. Hacer ejecutable el tiempo de evento

- Beam asocia un timestamp a cada elemento

- TimestampedValue asigna el tiempo del dominio
- WindowInto asigna cada elemento a una ventana lógica
- Las ventanas fijas no se superponen
- Las ventanas deslizantes responden preguntas solapadas
- Las sesiones agrupan actividad separada por inactividad
- La agregación ocurre por clave y por ventana
- WindowParam hace visible el intervalo en la salida
- Actividad: elige la ventana por su significado

3. Decidir cuándo un resultado es visible

- Una ventana define dónde; un trigger define cuándo
- El watermark estima el progreso del tiempo de evento

- AfterWatermark admite emisiones tempranas y tardías
- Cada firing produce un pane de la misma ventana
- Acumular o descartar cambia el significado de cada pane
- Allowed lateness limita cuánto tiempo aceptamos correcciones
- PaneInfo revela qué revisión estamos observando
- La política temporal es un contrato de producto
- TestStream reproduce tiempo, watermarks y desorden
- Actividad: define un contrato temporal

4. Recordar sin abandonar el modelo de Beam

- Los combiners bastan hasta que necesitamos control fino
- El estado está aislado por clave y por ventana

- Beam ofrece varias formas de estado tipado
- Un DoFn stateful declara la celda y su coder
- Deduplicar requiere recordar event_id observados
- Estado sin expiración se transforma en deuda operativa
- Las ventanas ayudan al garbage collection del estado

5. Hacer que una clave actúe más tarde

- Un timer programa un callback dentro del pipeline
- Event time y processing time responden preguntas distintas
- Un buffer por clave puede vaciarse al final de la ventana
- El callback lee, emite y limpia el buffer
- Estado y timer forman una pequeña máquina por clave

- Probar estado exige secuencias, no elementos aislados

6. Cruzar la frontera del pipeline

- Enriquecer no siempre requiere una llamada por evento
- Side input replica una vista pequeña y legible
- setup reutiliza clientes, pero no garantiza la escritura
- GroupIntoBatches reduce llamadas externas
- Un side effect puede repetirse aunque el código se vea lineal
- La clave de idempotencia representa el efecto lógico
- El identificador correcto depende del contrato de panes
- Separar cálculo y entrega reduce el acoplamiento
- Toda integración externa necesita una política explícita

- Actividad: encuentra el duplicado oculto

7. Laboratorio interactivo

- La práctica reproduce desorden, estado y reintentos
- El desafío integra las tres garantías
- Checklist de revisión del pipeline
- Ventana, estado y sink forman una sola promesa

8. Tarea 3 y cierre

- Tarea 3 — pipeline tolerante a desorden y reintentos
- Tarea 3 — entregables y criterios
- La tarea comienza en un notebook Marimo sin solución
- Fuentes y atribuciones

Del grafo al comportamiento temporal

Clase 5

- PCollection y PTransform
- Map, ParDo y combiners
- claves y shuffles
- ejecución local

Clase 6

- ventanas y panes
- datos tardíos
- estado por clave
- timers y side effects

Idea clave

Un pipeline de streaming correcto debe decidir qué recuerda, cuándo emite y cómo repite sin corromper el mundo externo.

Evento

event_id, merchant_id, event_time, amount, status

Ingesta

eventos fuera de orden



Beam

ventanas + estado



Salida

resumen idempotente

Pregunta analítica

¿Cuánto se confirmó por comercio y minuto, sin contar dos veces el mismo pago?

Desorden

Un pago de las 10:01 llega después de uno de las 10:03.

Duplicado

El productor reintentando y publica el mismo `event_id`.

Reintento

El worker escribe y falla antes de confirmar el progreso.

Idea clave

Ventanas, estado e idempotencia resuelven problemas diferentes; necesitamos las tres.

Al final de la clase deberías poder

- Asignar timestamps y ventanas con el SDK de Beam.
- Configurar triggers, lateness y modo de acumulación.
- Interpretar ventanas y panes en una salida.
- Implementar estado acotado por clave.
- Programar un timer y limpiar su estado.
- Elegir una estrategia de enriquecimiento.
- Diseñar side effects idempotentes o tolerantes a reprocesamiento.

¿Dónde?

¿A qué ventana pertenece el evento?

¿Cuándo?

¿Qué trigger hace visible el resultado?

¿Qué recordar?

¿Qué estado necesita cada clave?

¿Cómo escribir?

¿Qué evita duplicar el efecto externo?

Hacer ejecutable el tiempo de evento

Fuente de streaming

El conector suele producir elementos ya timestamped.

Fuente acotada

Si leemos un archivo, debemos extraer `event_time` y asignarlo.

Idea clave

Beam no puede inferir el tiempo del dominio a partir de un campo JSON: el pipeline debe convertirlo en timestamp.

Asignación explícita

```
from apache_beam.transforms.window import TimestampedValue

timestamped = events | "Use event time" >> beam.Map(
    lambda event: TimestampedValue(
        event,
        parse_iso(event["event_time"]).timestamp(),
    )
)
```

La unidad es segundos desde Unix epoch; un Timestamp de Beam también es válido.

Ventanas fijas de un minuto

```
windowed = timestamped | "Window per minute" >> beam.WindowInto(  
    beam.window.FixedWindows(60)  
)
```

La asignación todavía no agrega

La ventana se vuelve operativa cuando una transformación como GroupByKey o CombinePerKey reúne elementos.

10:00–10:01

e1 · e2

10:01–10:02

e3 · e4 · e5

10:02–10:03

e6

Semántica

Cada evento pertenece a exactamente un intervalo según su timestamp, no según el momento en que llegó.

Últimos 5 minutos, actualizado cada minuto

```
beam.WindowInto(  
    beam.window.SlidingWindows(size=300, period=60)  
)
```

Beneficio

Suaviza la señal y mantiene una vista móvil.

Costo

Un elemento puede contribuir a varias ventanas.

Gap de 10 minutos

```
beam.WindowInto(  
    beam.window.Sessions(gap_size=600)  
)
```

10:00

clic

+3 min

10:03

compra

+14 min

10:17

nueva sesión

Las ventanas de sesión pueden fusionarse cuando un evento tardío conecta dos grupos.

Total confirmado por comercio y minuto

```
totals = (  
  windowed  
  | "Only confirmed" >> beam.Filter(  
    lambda event: event["status"] == "CONFIRMED"  
  )  
  | "Key amount" >> beam.Map(  
    lambda event: (event["merchant_id"], event["amount"])  
  )  
  | "Sum per merchant and window" >> beam.CombinePerKey(sum)  
)
```

Idea clave

La clave reúne el comercio; la ventana limita el intervalo temporal.

Formatear el resultado

```
class FormatTotal(beam.DoFn):  
    def process(self, element, window=beam.DoFn.WindowParam):  
        merchant_id, total = element  
        yield {  
            "merchant_id": merchant_id,  
            "window_start": window.start.to_utc_datetime().isoformat(),  
            "window_end": window.end.to_utc_datetime().isoformat(),  
            "total": total,  
        }
```

El intervalo debe acompañar la métrica: sin él, “total = 350” pierde significado.

Pregunta	Ventana	Parámetro
Ventas por minuto calendario	¿?	60 s
Errores en los últimos 15 min cada 5 min	¿?	15 min / 5 min
Sesiones separadas por 30 min	¿?	gap 30 min

Decidir cuándo un resultado es visible

Window

Agrupar eventos por tiempo lógico.

+

Trigger

Produce uno o más panes observables.

Idea clave

El contenido de una ventana y el momento de publicarlo son decisiones independientes.

Antes

resultado temprano

Watermark cruza el fin

resultado on-time

Después

corrección tardía

Estimación, no reloj

El watermark expresa hasta qué tiempo el runner cree haber observado la mayor parte del input.

Contrato de emisión

```
from apache_beam.transforms import trigger

beam.WindowInto(
    beam.window.FixedWindows(60),
    trigger=trigger.AfterWatermark(
        early=trigger.AfterProcessingTime(10),
        late=trigger.AfterCount(1),
    ),
    allowed_lateness=120,
    accumulation_mode=trigger.AccumulationMode.ACCUMULATING,
)
```

Temprano: baja latencia. On-time: watermark. Tardío: corrige al llegar cada nuevo elemento.

EARLY

total = 100
estimación rápida

ON_TIME

total = 170
cierre según watermark

LATE

total = 190
corrección

Idea clave

La salida no es necesariamente “una fila final”; puede ser una secuencia de revisiones.

Modo	Panes	Lectura
Acumulando	100 → 170 → 190	cada pane reemplaza al anterior
Descartando	100 → 70 → 20	cada pane es un delta

Contrato del sink

El consumidor debe saber si recibe revisiones completas o incrementos.

Dentro del límite

El evento puede actualizar la ventana y producir un pane tardío.

Después del límite

El evento es demasiado tardío para esa política y puede descartarse.

Idea clave

Más lateness mejora completitud, pero prolonga estado, costo y revisiones.

Metadatos de emisión

```
class AttachPane(beam.DoFn):  
    def process(  
        self,  
        element,  
        pane=beam.DoFn.PaneInfoParam,  
    ):  
        yield {  
            "value": element,  
            "pane_index": pane.index,  
            "timing": str(pane.timing),  
            "is_first": pane.is_first,  
            "is_last": pane.is_last,  
        }
```

Latencia

¿Cuán pronto necesita una estimación el usuario?

Complejidad

¿Cuánto atraso real debemos tolerar?

Costo

¿Cuántas revisiones y cuánto estado son aceptables?

Idea clave

No existe un trigger universal: la decisión depende del uso del resultado.

Fuente determinista para pruebas

```
stream = (  
    TestStream()  
    .advance_watermark_to(0)  
    .add_elements([  
        TimestampedValue(("m1", 100), 5),  
        TimestampedValue(("m1", 70), 40),  
    ])  
    .advance_watermark_to(60)  
    .add_elements([TimestampedValue(("m1", 20), 50)])  
    .advance_watermark_to_infinity()  
)
```

El último elemento llega después de que el watermark cruzó el fin, pero conserva timestamp 50.

Alerta de fraude por comercio

La alerta debe aparecer en menos de 15 segundos; el 99 % de los eventos llega antes de 2 minutos; una corrección completa es preferible a un delta.

Decidan y justifiquen

Tamaño de ventana · early trigger · late trigger · allowed lateness · accumulation mode.



10 minutos

Al volver: memoria por clave, timers y efectos externos.

**Recordar sin abandonar el modelo
de Beam**

Usar una transformación estándar

suma, conteo, promedio, top-N, agrupación o batching soportado.

Considerar estado explícito

deduplicación, máquinas de estados, buffers y lógica secuencial por clave.

Idea clave

El estado de usuario es una herramienta de último kilómetro, no un reemplazo automático de `CombinePerKey`.

Clave m1 · ventana A

estado independiente

Clave m2 · ventana A

estado independiente

Clave m1 · ventana B

estado independiente

Clave m2 · ventana B

estado independiente

La entrada a un ParDo stateful debe ser una PCollection de pares clave–valor.

Spec Python	Semántica	Ejemplo
ReadModifyWriteStateSpec	un valor	último estado
BagStateSpec	colección no ordenada	buffer
SetStateSpec	valores únicos	IDs observados
CombiningValueStateSpec	acumulador combinado	suma o conteo

Referencia: [Apache Beam Python API — userstate](#).

Contador por clave

```
from apache_beam.coders import VarIntCoder
from apache_beam.transforms.userstate import ReadModifyWriteStateSpec

class CountSeen(beam.DoFn):
    COUNT = ReadModifyWriteStateSpec("count", VarIntCoder())

    def process(
        self,
        element,
        count=beam.DoFn.StateParam(COUNT),
    ):
        key, value = element
        next_count = (count.read() or 0) + 1
        count.write(next_count)
        yield key, next_count
```

Estado Set por comercio

```
from apache_beam.coders import StrUtf8Coder
from apache_beam.transforms.userstate import SetStateSpec

class Deduplicate(beam.DoFn):
    SEEN = SetStateSpec("seen_ids", StrUtf8Coder())

    def process(self, element, seen=beam.DoFn.StateParam(SEEN)):
        merchant_id, event = element
        event_id = event["event_id"]
        if event_id not in set(seen.read()):
            seen.add(event_id)
            yield merchant_id, event
```

La clave decide el dominio de deduplicación; la ventana y la limpieza limitan cuánto recordamos.

Crecimiento

IDs y buffers ocupan memoria o disco del runner.

Recuperación

Más estado implica checkpoints más pesados.

Semántica

Borrar demasiado pronto permite duplicados antiguos.

Idea clave

Toda celda de estado necesita una política explícita de acotación o limpieza.

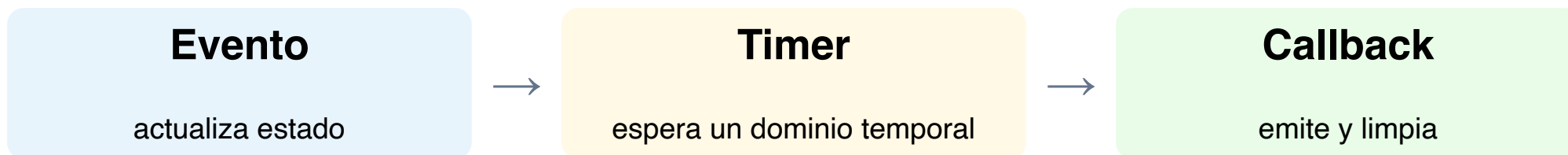
Regla del modelo

El estado por clave está también limitado por ventana. El runner puede recolectarlo cuando el watermark supera el fin de ventana más la lateness permitida.

GlobalWindow

Si el estado vive en la ventana global, necesitamos timers u otra estrategia explícita de expiración.

Hacer que una clave actúe más tarde



Idea clave

El timer pertenece a la misma clave y ventana que el estado que coordina.

WATERMARK

Actúa cuando progresa el tiempo de evento.

Útil para cerrar lógica vinculada al dominio.

REAL_TIME

Actúa según el reloj de procesamiento.

Útil para timeout o flush operativo.

La disponibilidad exacta y el comportamiento operativo dependen del runner.

Declaración

```
from apache_beam.transforms.timeutil import TimeDomain
from apache_beam.transforms.userstate import BagStateSpec, TimerSpec

class FlushPayments(beam.DoFn):
    BUFFER = BagStateSpec("buffer", beam.coders.PickleCoder())
    FLUSH = TimerSpec("flush", TimeDomain.WATERMARK)

    def process(
        self,
        element,
        window=beam.DoFn.WindowParam,
        buffer=beam.DoFn.StateParam(BUFFER),
        flush=beam.DoFn.TimerParam(FLUSH),
    ):
        _, payment = element
        buffer.add(payment)
        flush.set(window.end)
```

Callback

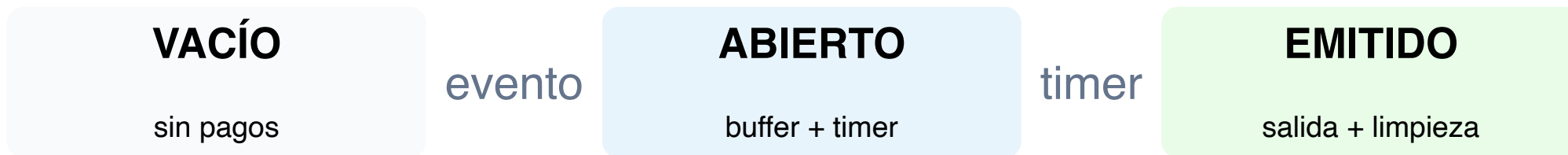
```
from apache_beam.transforms.userstate import on_timer

class FlushPayments(beam.DoFn):
    # ... BUFFER y FLUSH ...

    @on_timer(FLUSH)
    def on_flush(
        self,
        buffer=beam.DoFn.StateParam(BUFFER),
        key=beam.DoFn.KeyParam,
    ):
        payments = list(buffer.read())
        buffer.clear()
        if payments:
            yield key, payments
```

Idea clave

Emitir sin limpiar repite el buffer; limpiar antes de diseñar la salida puede perder contexto ante fallos.



Invariante

Para cada clave, todo evento aceptado está en el buffer o ya está representado en una salida.

Misma clave

dos eventos comparten estado

Claves distintas

los estados no se mezclan

Duplicado

el segundo evento no vuelve a emitirse

Timer

la salida aparece y el estado se limpia

`TestStream` permite controlar eventos y watermark; `assert_that` verifica las salidas.

Cruzar la frontera del pipeline

Patrón	Cuándo	Trade-off
Side input	referencia pequeña	snapshot replicado
CoGroupByKey	dos colecciones grandes	shuffle
Lookup externo	dato dinámico remoto	latencia y fallos
Cache por worker	lecturas repetidas	frescura y memoria

Catálogo de comercios

```
merchant_view = beam.pvalue.AsDict(  
    merchants | beam.Map(lambda m: (m["id"], m))  
)  
  
enriched = payments | beam.Map(  
    lambda payment, catalog: {  
        **payment,  
        "category": catalog[payment["merchant_id"]]["category"],  
    },  
    catalog=merchant_view,  
)
```

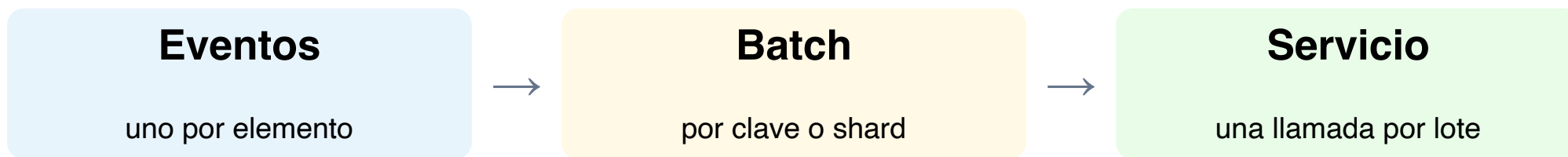
La vista debe ser suficientemente pequeña; también debemos definir cómo se actualiza.

Ciclo de vida por instancia

```
class LookupRisk(beam.DoFn):  
    def setup(self):  
        self.client = RiskClient()  
  
    def process(self, payment):  
        score = self.client.score(payment)  
        yield {**payment, "risk_score": score}  
  
    def teardown(self):  
        self.client.close()
```

Idea clave

Reutilizar conexiones mejora eficiencia; no elimina timeouts, reintentos ni respuestas duplicadas.



Trade-off

Menos llamadas y mayor throughput a cambio de espera, memoria y manejo de resultados parciales.

1. Escribir

la API acepta



2. Fallar

no se confirma progreso



3. Reintentar

la API recibe otra vez

Idea clave

“Exactly once” dentro del runner no convierte automáticamente una API externa en exactly once.

Upsert del resumen

```
idempotency_key =  
    merchant_id + window_start
```

Operación externa

```
PUT /merchant-window/{idempotency_key} O UPSERT ... ON CONFLICT DO  
UPDATE
```

Cada revisión y cada reintento reemplazan la misma entidad lógica en vez de crear otra.

Revisión acumulativa

Clave: comercio + ventana.

Cada pane reemplaza el total anterior.

Delta descartando

Clave: comercio + ventana + pane.

Cada pane contribuye una sola vez.

Idea clave

La idempotencia no es solo técnica: codifica qué significa “el mismo efecto”.



Ventaja

El pipeline no bloquea su progreso en una API frágil; la entrega puede reintentarse y observarse por separado.

Timeout

¿Cuánto esperar antes de declarar fallo?

Retry

¿Qué errores son transitorios y con qué backoff?

Idempotencia

¿Qué clave identifica el efecto lógico?

Dead letter

¿Dónde queda lo que no pudo procesarse?

Código

```
class SendAlert(beam.DoFn):  
    def process(self, alert):  
        requests.post("/alerts", json=alert).raise_for_status()  
        yield alert
```

Discusión

¿Qué ocurre si el servidor creó la alerta, pero la respuesta se pierde?
Propongan una clave y un contrato idempotente.

Laboratorio interactivo

Notebook Marimo

- explorar eventos y atraso
- ajustar ventana y lateness
- ejecutar un pipeline Beam
- deduplicar por estado
- simular un sink idempotente

Repositorio público

github.com/rparrapy/streaming-fpuna-clase6

```
git clone https://github.com/rparrapy/streaming-fpuna-clase6.git
cd streaming-fpuna-clase6
docker compose up --build notebook
```

Objetivo

Producir totales confirmados por comercio y minuto, tolerando 120 segundos de atraso y sin duplicar pagos ni escrituras.

Temporal

ventana + pane + lateness

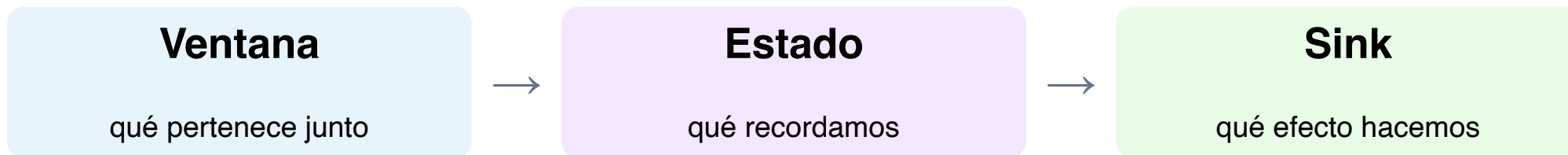
Estado

deduplicación por event_id

Externa

upsert con clave idempotente

- ☐ El timestamp proviene del tiempo de evento.
- ☐ La ventana expresa la pregunta analítica.
- ☐ El trigger y los panes tienen un contrato documentado.
- ☐ La lateness corresponde a datos observados o a un SLA.
- ☐ El estado está aislado y acotado.
- ☐ Los timers limpian o renuevan el estado deliberadamente.
- ☐ La salida tolera reintentos.
- ☐ Las pruebas incluyen desorden, duplicados y fallo externo.



Idea clave

La corrección end-to-end depende de que las tres decisiones compartan la misma semántica.

Tarea 3 y cierre

Entrega individual — 15%

Completar un pipeline de pagos con tiempo de evento, ventanas, estado por clave, expiración e idempotencia.

Contrato obligatorio

- totales CONFIRMED por comercio y minuto
- hasta 120 segundos de lateness
- deduplicación por event_id dentro del comercio
- panes acumulativos y metadatos de ventana
- timer que limite la vida del estado
- UPSERT por merchant_id|window_start

Idea clave

La entrega debe demostrar qué ocurre con desorden, duplicados, atraso y reintentos; no basta con mostrar el caso feliz.

Entregables

- notebook implementado
- pruebas reproducibles
- evidencia de ejecución
- README con decisiones
- instrucciones Docker o uv

Evaluación

- tiempo y ventanas: 25%
- estado y expiración: 25%
- idempotencia: 20%
- pruebas: 20%
- reproducibilidad: 10%

Pruebas mínimas

Duplicado, claves aisladas, evento fuera de orden, late aceptado, evento demasiado tardío, timer de limpieza y escritura repetida.

Esqueleto

- consignas dentro del notebook
- ocho funciones o métodos pendientes
- dataset de pagos
- Docker y lockfile reproducible
- firmas y contratos, sin implementación

Repositorio público

github.com/rparray/streaming-fpuna-clase6-tarea

```
git clone https://github.com/rparray/streaming-fpuna-clase6-tarea.git
cd streaming-fpuna-clase6-tarea
docker compose up --build notebook
```

- Ventanas, timestamps, triggers, panes y lateness: [Apache Beam Programming Guide — Windowing y Triggers](#).
- Estado, timers y garbage collection: [Apache Beam Programming Guide — State and Timers](#).
- API Python de estado: [Apache Beam Python API — userstate](#).
- Pruebas con tiempo simulado: [Apache Beam — TestStream](#).
- Batching para servicios externos: [Apache Beam patterns](#).

Material de clase

El caso de pagos, sus datos y el laboratorio Marimo fueron elaborados para esta práctica.

¿Preguntas?

 rmparra@pol.una.py

 Laboratorio: Beam avanzado en Marimo

 Próxima clase: aplicación end-to-end con Kafka y Beam